

An Area-Efficient Iterative Single-Precision Floating-Point Multiplier Architecture for FPGA

Sunwoong Kim

University of Illinois at Urbana-Champaign
Urbana, Illinois 61801 USA
sunwoong@illinois.edu

Rob A. Rutenbar

University of Pittsburgh
Pittsburgh, Pennsylvania 15260 USA
rutenbar@pitt.edu

ABSTRACT

Approximate multipliers have been widely used in critical applications, such as machine learning and multimedia, which are tolerant to approximation errors. This paper proposes a novel single-precision floating-point (SPFP) multiplication algorithm and its architecture. The proposed work approximates only one of the operands to reduce the number of logic blocks and iteratively compensates the approximation error to achieve acceptable error ranges in applications. To reduce the accuracy degradation by the single operand approximation, a rounding scheme and an operand selection scheme are additionally introduced. Compared with the widely-known previous iterative Mitchell design, our proposed SPFP multiplier design decreases the numbers of look up tables (LUTs) and flip flops (FFs) by 55% and 59% respectively, and shows two cycles shorter latency. The accuracy of our design becomes close to that of the iterative Mitchell design as the number of iterations increases, and it always meets the error tolerance of 1% when the number of iterations is four.

KEYWORDS

Approximate computing, floating-point multiplier, FPGA

ACM Reference format:

Sunwoong Kim and Rob A. Rutenbar. 2019. An Area-Efficient Iterative Single-Precision Floating-Point Multiplier Architecture for FPGA. In *Proceedings of Great Lakes Symposium on VLSI 2019, Tysons Corner, VA, USA, May 9–11, 2019 (GLSVLSI '19)*, 6 pages.
<https://doi.org/10.1145/3299874.3318002>

1 INTRODUCTION

Since floating-point (FP) arithmetic provides a wider dynamic range than fixed-point arithmetic, specific applications such as scientific computations often use FP operators. Even in machine learning applications where fixed-point numbers are widely used, training still requires much more precision [1]. Nevertheless, the FP operators have critical issues such as significant circuit area, long latency, and excessive power consumption. In general, FP multiplication is one of the most common and expensive arithmetic operations [2], [3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GLSVLSI '19, May 9–11, 2019, Tysons Corner, VA, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6252-8/19/05...\$15.00

<https://doi.org/10.1145/3299874.3318002>

To alleviate the critical issues, *approximate multipliers* have been widely used in critical applications, such as machine learning and multimedia, which are tolerant to the approximation error [1]–[7]. The error tolerance is due to various reasons such as the imperfection of human sensory systems and data redundancy, and the approximation error often does not significantly impair the overall performance of the applications [4].

Typically, acceptable error ranges of each application is different, and therefore approximate multipliers with fixed accuracy have limitations [5]. To resolve this issue, multiplication algorithms that enable us to configure accuracy while trading for area, latency, and/or power consumption have been presented [2], [5], [6]. Zhang *et al.* apply Mitchell's algorithm [8] to the mantissa multiplication of FP numbers and design to configure the accuracy [5]. However, there are only two configuration modes, and the errors always occur due to Mitchell's algorithm. The configurable FP multiplier proposed by Imani *et al.* discards one of the input mantissas and directly uses the other when the discarded mantissa does not exceed a threshold [2]. However, when the discarded mantissa exceeds the threshold, the existing mantissa multiplication is still executed. Babic *et al.* present a multiplier architecture based on Mitchell's algorithm [6]. Since it iteratively compensates approximation errors using the same circuits, various accuracy levels are provided. However, the architecture is only for integer multiplication, and therefore an optimized architecture for FP numbers is still needed.

The main contributions of this paper are as follows:

- We present an iterative single-precision FP (SPFP) multiplier architecture where the appropriate tradeoff between accuracy and latency is configurable.
- We approximate only a single operand to reduce the number of logic blocks, and introduce novel rounding and operand selection schemes to reduce the accuracy degradation.

The rest of this paper is organized as follows: Section 2 reviews background for this paper. Our proposed iterative SPFP multiplier architecture is presented in Section 3 and evaluated in Section 4. Lastly, Section 5 concludes this paper.

2 BACKGROUND

In the IEEE 754 SPFP format [9], each number is represented by 32-bits, which is composed of 1-bit for sign, 8-bits for exponent, and 23-bits for mantissa. The mantissa has a hidden most significant bit that is always set to 1. An SPFP number x is represented as follows:

$$x = \pm 1.m \times 2^{e-127}, \quad 0 \leq m < 1 \quad (1)$$

where m and e represent mantissa and exponent, respectively.

Typically, conventional SPFP multipliers generate results in the following order: 1) Add exponents of two numbers and then subtract

it by the bias that is 127; 2) Multiply mantissas; 3) Calculate the sign by XORing [10]. Since the second step accounts for the bulk of hardware resources, many studies have focused on approximation of the mantissa multiplication [3], [5], [11]. However, in general, the error generated by this approach increases exponentially as the number of truncated bits increases.

Mitchell's algorithm is in the family of logarithmic multiplication methods, which is advantageous in terms of accuracy compared with other approximate methods [8]. First, it calculates logarithms of two operands using (2).

$$\log_2 x = l + \log_2(1 + r) \quad (2)$$

where $l = \lfloor \log_2 x \rfloor$ and r is a rational number satisfying $0 \leq r < 1$. The multiplication of two numbers is then represented using (3).

$$\begin{aligned} x_1 \cdot x_2 &= 2^{l_1+l_2} \cdot (1 + r_1) \cdot (1 + r_2) \\ &= 2^{l_1+l_2} \cdot (1 + r_1 + r_2 + r_1 \cdot r_2) \end{aligned} \quad (3)$$

To substitute the multiplication with a shift and additions, the term of $r_1 \cdot r_2$ is omitted from (3), and the multiplication is approximated as follows:

$$x_1 \cdot x_2 \approx \begin{cases} 2^{l_1+l_2} \cdot (1 + r_1 + r_2), & r_1 + r_2 < 1 \\ 2^{l_1+l_2+1} \cdot (r_1 + r_2), & r_1 + r_2 \geq 1 \end{cases} \quad (4)$$

When the discarded $r_1 \cdot r_2$ is large, Mitchell's algorithm produces a sizeable error. To reduce the error, an iterative scheme was proposed [6]. In this scheme, the multiplication is represented as follows:

$$\begin{aligned} x_1 \cdot x_2 &= (2^{l_1} + x_1^{[1]}) \cdot (2^{l_2} + x_2^{[1]}) \\ &= 2^{l_1+l_2} + 2^{l_1} \cdot x_2^{[1]} + 2^{l_2} \cdot x_1^{[1]} + x_1^{[1]} \cdot x_2^{[1]} \end{aligned} \quad (5)$$

where the number in square brackets represents the current iteration index and $x^{[1]} = x - 2^l$. During the first iteration, the first three terms of (5) compose the partial result $P_1(x_1 \cdot x_2)$ that is implemented using shifters and adders. During the second iteration, the last term of (5), which is the first iteration *error*, is approximated with the same way as follows:

$$x_1^{[1]} \cdot x_2^{[1]} = P_2(x_1 \cdot x_2) + x_1^{[2]} \cdot x_2^{[2]} \quad (6)$$

The approximate multiplication result up to the second iteration is the sum of $P_1(x_1 \cdot x_2)$ and $P_2(x_1 \cdot x_2)$. The generalized formula is shown in (7).

$$x_1 \cdot x_2 \approx \sum_{i=1}^n P_i(x_1 \cdot x_2) \quad (7)$$

As the number of iterations n increases, the number of partial results increases and the error is reduced. Since it enables us to trade off latency against error on the same circuit, this scheme can be used efficiently in various applications.

Fig. 1 shows an architecture of the basic block for iterative integer multiplication [6]. The basic block is based on a 4-stage pipeline, and therefore generates a partial result after 4 cycles. It comprises 2 leading one detectors (LODs), 2 shifters, 3 adders, and other components. Note that the adders in the first stage can be implemented using XOR gates, so they are not counted. To accumulate the partial results, one more adder and register are required. Since Babic *et al.* only present the iterative multiplier architecture for integers [6], an architecture for SPFP numbers is proposed in this paper.

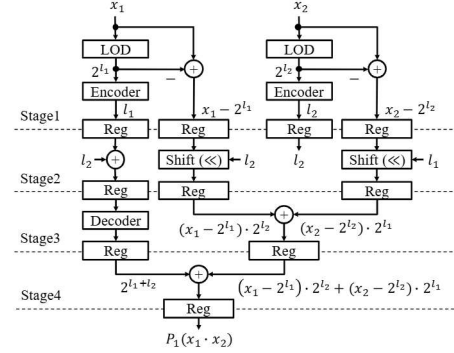


Figure 1: Architecture of the iterative integer multiplier [6].

Algorithm 1: Approximate operand with a rounding scheme

```

1 error ← magn(x) and ap ← 0
2 for k = 1 to n do
3   if error = 0 then
4     Break
5   else
6     f ← ⌊log2|error| - ⌊log2|error|⌋
7     l ← f × ⌊log2|error|⌋ + (1 - f) × ⌊log2|error|⌋
8     if error < 0 then
9       error ← error + 2l
10      ap ← ap - 2l
11     else
12       error ← error - 2l
13       ap ← ap + 2l
14     end
15   end
16 end
17 APn(x) ← sign(x) × ap

```

3 ITERATIVE SPFP MULTIPLIER

This section proposes a novel iterative SPFP multiplication algorithm and its architecture.

3.1 Rounding Scheme

The most intuitive scheme (we call it a *baseline*) is to iteratively add power-of-two numbers that are the closest to and less than an error ($= x_1 \cdot x_2 - \sum P_i(x_1 \cdot x_2)$). When the number of iterations is n , the shift amount $l_n(\cdot)$ and the approximate value on input x are calculated using (8) and (9), respectively.

$$l_n(x) = \lfloor \log_2(|x| - \sum_{i=1}^n 2^{l_{i-1}(x)}) \rfloor \quad (8)$$

$$AP_n(x) = \text{sign}(x) \times \sum_{i=1}^n 2^{l_i(x)} \quad (9)$$

In (8), the initial $2^{l_0(x)}$ is 0. Even though the errors are compensated by this scheme, their values are still large. To reduce these errors, we propose a rounding scheme, which is shown in Algorithm 1. First of all, a fraction of $\log_2|\text{error}|$ is calculated (line 6). If its round number f is 1, a shift amount is set to $\lfloor \log_2|\text{error}| \rfloor$. Otherwise,

Table 1: Approximation example of SPFP number 0.4656868*

n	$f_n(x)$	$l_n(x)$	$t_n(x)$	$z_n(x) + 1$	Normalization			$AP_n(x)$
					$m_{n+1}(x)$	$e_{n+1}(x)$	Next $ error $	
1	1	$125 - 126 = -1$	$(0).0010001100...1_2$	3	$(1).0001100...0_2$	$125 - 3 = 122$	0.03431320	0.50000000
2	0	$122 - 127 = -5$	$(0).0001100...0_2$	4	$(1).100...0_2$	$122 - 4 = 118$	0.00306320	0.46875000
3	1	$118 - 126 = -8$	$(0).011...1_2$	2	$(1).1...0_2$	$118 - 2 = 116$	0.00084305	0.46484375

* $e_1(0.4656868) = 125$ and $m_1(0.4656868) = (1).1101110011011010000000_2$

$\lceil \log_2 |error| \rceil$ is used for the shift amount (line 7). In (8), the error ($= |x| - \sum 2^{l_i(x)}$) is always zero or a positive number. However, the error in Algorithm 1 can be a negative number as well. Therefore, the error and the intermediate approximate value ap are updated differently from the baseline. Specifically, if the error is a negative (positive) number, the error is added (subtracted) by 2^l and ap is subtracted (added) by 2^l (lines 8-14). After all iterations, the output value is calculated by multiplying ap and the sign of x (line 17).

The proposed approximate algorithm processes SPFP numbers represented by the IEEE 754 SPFP format [9], which makes the architecture different from that for integer numbers. First of all, we propose a scheme to efficiently calculate the f in Algorithm 1. The integer portion of binary logarithm is only related to the exponent of an FP argument. On the other hand, the fractional portion is influenced by the mantissa. Therefore, we can know whether the fractional part of $\log_2 |error|$ in Algorithm 1 is larger than or equal to 0.5 using the mantissa. As an argument increases, its binary logarithm increases. In addition, the argument satisfying that its binary logarithm is 0.5 is $\sqrt{2}$, and the mantissa of $\sqrt{2}$ is $(1).0110...11_2$. Therefore, when the mantissa of an FP number is larger than or equal to $(1).0110...11_2$, f is set to 1. To make the circuit simpler, we use $(1).1000...00_2$ instead of the $(1).0110...11_2$, which means that the 22nd bit from the least significant bit of the mantissa is used as f . The approximate f has little effect on accuracy.

As shown in Algorithm 1, rounding-up occurs when f is 1. In this case, l is increased by 1 and the absolute value of the next error is not $|error| - 2^l$ but $2^l - |error|$. In other words, l and the next error are calculated differently depending on the f value. When $f_n(x)$ represents the f for an SPFP input x and the iteration number n , a shift amount is calculated using (10).

$$l_n(x) = \begin{cases} e_n(x) - 127, & \text{if } f_n(x) = 0 \\ e_n(x) - 126, & \text{otherwise} \end{cases} \quad (10)$$

where $e_n(x)$ represents the exponent during the n -th iteration. Suppose that $m_n(x)$ represents the mantissa during the n -th iteration, and $t_n(x)$ refers to the mantissa of the difference between the error and $2^{l_n(x)}$ before normalization. If $f_n(x)$ is 0, $t_n(x)$ is set to $m_n(x)$ except that the hidden bit becomes zero (see the third row of Table 1). On the other hand, if $f_n(x)$ is 1, $t_n(x)$ is set to $-m_n(x)$ (see the second and fourth rows of Table 1). Note that $-m_n(x)$ is approximately implemented using a complement operator in this paper. The update (or normalization) processes for mantissa and exponent are shown in (11) and (12), respectively.

$$m_{n+1}(x) = t_n(x) \times 2^{(z_n(x)+1)} \quad (11)$$

$$e_{n+1}(x) = e_n(x) - (z_n(x) + 1) \quad (12)$$

where $z_n(x)$ refers to the number of bit 0s before the first bit 1 appears in $t_n(x)$. Since the hidden bit of $t_n(x)$ is always zero, the mantissa and exponent are normalized using " $z_n(x) + 1$ ".

Table 1 shows an approximation example for the SPFP number 0.4656868. In this table, binary numbers are represented by subscripts. The initial $e_1(0.4656868)$ and $m_1(0.4656868)$ values are 125 and $(1).1101110011011010000000_2$, respectively. Since $f_1(0.4656868)$ is 1, $l_1(0.4656868)$ is calculated as -1 using (10). The first bit 1, represented in bold, appears at the 20th bit of $t_1(0.4656868)$, $z_1(0.4656868)$ is set to 2. Using (11) and (12), $m_2(0.4656868)$ and $e_2(0.4656868)$ are set to $(1).0001100...0_2$ and 122, respectively. During the second iteration, $l_2(0.4656868)$ is calculated as -5. Meanwhile, $z_2(0.4656868)$ is set to 3 because the first bit 1 appears at the 19th bit of $t_2(0.4656868)$. Consequently, $m_3(0.4656868)$ and $e_3(0.4656868)$ are set to $(1).100...0_2$ and 118, respectively. The $e_3(0.4656868)$ sets $l_3(0.4656868)$ to -8. After the iteration number of 3, the SPFP number of 0.4656868 is approximated to 0.46484375 ($= 2^{-1} - 2^{-5} - 2^{-8}$). Note that the baseline approximates 0.4656868 to 0.4375 ($= 2^{-2} + 2^{-3} + 2^{-4}$).

3.2 Operand Selection Scheme

Since our multiplier approximates only a *single* operand, its architecture is simpler than the iterative Mitchell. On the other hand, it cannot beat the iterative Mitchell in accuracy because one more term in (5) is basically omitted. However, the accuracy degradation can be reduced by *choosing* the operand that has lesser approximation error.

If our rounding scheme is not used, it is easy to calculate the error because $n - 1$ ones in the mantissa always survive (e.g., the mantissa of the approximated 0.4375, presented in the previous section, is $(1).110...0_2$ and the error starts from the third one of $(1).1101110011011010...0_2$). However, when our rounding scheme is applied, it is hard to know how many ones survive in advance (e.g., the mantissa of the approximated 0.46484375 is $(1).1101110...0_2$), which makes the error prediction difficult.

To resolve this issue, regular bit-patterns in the mantissa are analyzed and utilized in the proposed design. Table 2 shows the mantissa prediction table when n is three. In the second column, RD and RU are abbreviations of rounding-up and rounding-down, respectively, and G_i represents the i -th group of successive 1s. In the third and fourth columns, "0..." and "1..." represent successive 0s and 1s respectively, and * refers to the remaining bits (don't-care term). There are four bit-patterns checked in this table: 010, 011, 101, and 100. As explained in the previous section, bits in the mantissa t_n are flipped when rounding-up occurs. If the bits are flipped an even number of times, the next 010 or 011 pattern is searched. The middle 1 of the pattern becomes a hidden bit and the next bit is used as the next f after normalization. On the other hand, if the bits are flipped

Table 2: Mantissa prediction table when n is three

(f_1, f_2, f_3)	Function	Input mantissa	Output mantissa
(0, 0, 0)	RD @G3	(1).0...010...010* ₂	(1).0...010...010...0 ₂
(0, 0, 1)	RU @G2	(1).0...010...011* ₂	(1).0...010...100...0 ₂
(0, 1, 0)	RU @G2	(1).0...011...101* ₂	(1).0...011...110...0 ₂
(0, 1, 1)	RD @G2	(1).0...011...100* ₂	(1).0...011...100...0 ₂
(1, 0, 0)	RU @G3	(1).1...101...101* ₂	(1).1...101...110...0 ₂
(1, 0, 1)	RD @G3	(1).1...101...100* ₂	(1).1...101...100...0 ₂
(1, 1, 0)	RD @G3	(1).1...100...010* ₂	(1).1...100...010...0 ₂
(1, 1, 1)	RU @G2	(1).1...100...011* ₂	(1).1...100...100...0 ₂

	Ex1. Mantissa of 0.4656868	Ex2. Mantissa of 0.3361206
Input mantissa:	(1). <u>11011100</u> 110111010000000	(1). <u>0101100000</u> 1100000000000
	G1 G2 G3 G4 G5	G1 G2 G3
	↓ Rounding-down	↓ Rounding-up
Output mantissa:	(1).110111000000000000000000	(1).011000000000000000000000
[Error]:	(0).000000001101110100000000 <	(0).000001111101000000000000

Figure 2: Example of mantissa prediction on two operands.

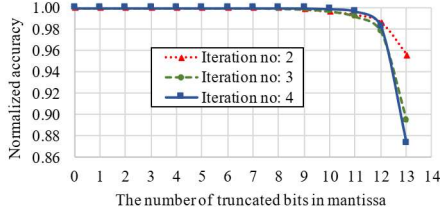


Figure 3: Normalized accuracy depending on the number of truncated bits in mantissa.

an odd number of times, 101 or 100 pattern is searched because a bit-flip changes 010 and 011 into 101 and 100, respectively. Fig. 2 shows an example of the mantissa prediction on two operands using Table 2. The mantissa of 0.4656868 satisfies the seventh row of Table 2, and the output mantissa is (1).1101110...0₂, which is the same result of Table 1. On the other hand, the mantissa of 0.3361206 satisfies the third row of Table 2, and the output mantissa is (1).0110...0₂. Since the absolute value of the error of 0.4656868 is smaller than that of 0.3361206, 0.4656868 is selected as the operand to approximate. Note that the error of 0.3361206 is smaller than that of 0.4656868 in the baseline, which makes the opposite selection result.

This operand selection scheme assures the almost-optimal result but requires rather many logic blocks and a long path delay. Especially, it needs multiple mantissa prediction tables for each iteration number because functions (and output mantissas) of the tables are different. To alleviate the hardware-related issue, two additional sub-schemes are proposed. First, only a *single* type of table is deployed while sacrificing accuracy. In our design, a prediction table for n of 2 is empirically chosen. The details are explained in Section 4.1. Second, mantissas of input operands are truncated, which affects the number of logic blocks and the critical path delay of the entire design as well. Fig. 3 shows normalized accuracy depending on the number of truncated bits. The normalized accuracy

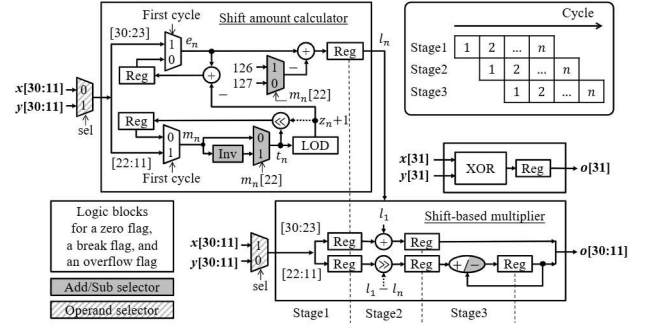


Figure 4: Overall architecture of our SPFP multiplier.

is calculated as the ratio of multiplications satisfying error $< 0.1\%$ with truncation to that without truncation. Since the normalized accuracy is larger than 0.99 until an 11-bit truncation, our design truncates the least significant 11-bits of two input operands from the beginning. To further reduce the number of logic blocks and the path delay, the least significant 5-bits of input mantissas for the prediction table are additionally truncated (the most significant 7-bits of input mantissas are used for prediction), which sacrifices the prediction accuracy. The details are described in Section 4.1.

3.3 Overall Architecture

Fig. 4 shows the overall architecture of our iterative SPFP multiplier where blocks represented by grey color and diagonal lines refer to additional logic blocks for the proposed rounding scheme and operand selection scheme, respectively. The detailed logic blocks for a *zero* flag (when input operand(s) is(are) zero), a *break* flag (when the intermediate mantissa becomes zero), and an *overflow* flag (when an overflow occurs in the adder/subtractor; the exponent increases or decreases by 1 and the mantissa is additionally right- or left-shifted by 1) are omitted for the sake of simplicity. The proposed architecture is based on a 3-stage pipeline. In Stage1, the sign bit is simply calculated using an XOR gate and stored in a register. In addition, the operand to approximate is chosen by the operand selector. The shift amount calculator generates l values of the selected operand using (10), (11), and (12). In the shift-based multiplier, the exponent and mantissa of the other operand are stored in registers. In Stage2, the shift-based multiplier calculates the exponent and mantissa corresponding the l values. As the number of iterations increases, the number multiplied by 2^l has a smaller exponent. Since mantissas are aligned based on the mantissa with the largest exponent, the first l_1 value is only added to the exponent and the mantissa is aligned by a shifter with the shift amount of " $l_1 - l_n$ ". In Stage3, the shifted mantissa is added to or subtracted from the accumulated mantissa by an integer adder/subtractor. A selecting signal for the adder/subtractor is generated in the add/sub selector, and it is reset to 0 (Add) and flipped whenever $f_n(x)$ is 1.

For example, to calculate $0.4656868 \times 0.3361206$ with the iteration number of 3, 0.4656868, which is selected as shown in Fig. 2, generates l values of -1, -5, and -8 in the shift amount calculator (see the third column of Table 1). The numbers of 0.3361206, shifted by the l values in the shift-based multiplier, are 0.1680603, 0.01050376875, and 0.00131297109375, respectively. To match the exponents of

Table 3: Accuracy comparison of previous and proposed SPFP approximate multipliers with 200 million inputs

n	Algorithm	Correct (%)				
		Error < 0.1%	Error < 0.5%	Error < 1.0%	Error < 5.0%	Error < 10.0%
1	Mitchell [8]	0.99	4.17	7.69	31.22	56.25
2	Iterative Mitchell [6]	19.14	47.37	65.13	99.13	100.00
	Proposed BASE	1.20	4.80	8.62	32.24	55.56
	Proposed BASE+RND	4.97	17.70	29.46	79.32	100.00
	Proposed BASE+RND+TRNC+SEL	9.30	32.12	50.01	95.58	100.00
3	Iterative Mitchell [6]	70.52	95.52	99.43	100.00	100.00
	Proposed BASE	6.36	19.79	31.06	75.00	94.45
	Proposed BASE+RND	28.92	65.89	84.72	100.00	100.00
	Proposed BASE+RND+TRNC+SEL	43.94	83.01	95.03	100.00	100.00
4	Iterative Mitchell [6]	98.03	100.00	100.00	100.00	100.00
	Proposed BASE	20.19	47.57	64.17	97.37	100.00
	Proposed BASE+RND	74.00	99.25	100.00	100.00	100.00
	Proposed BASE+RND+TRNC+SEL	86.69	99.81	100.00	100.00	100.00

Error (%) = $100 \times |\text{ideal result} - \text{approximate result}| / \text{ideal result}$

BASE: Baseline, RND: Rounding scheme, TRNC: Truncation, SEL: Operand selection scheme

these three numbers to 124 that is the biggest value among them, mantissas of 0.01050376875 and 0.00131297109375 are right-shifted by 4 ($= (-1) - (-5)$) and 7 ($= (-1) - (-8)$), respectively. Then, the shifted mantissas are added to or subtracted from the accumulated value depending on the selecting signal.

4 EVALUATION

This section evaluates the proposed iterative SPFP multiplier in terms of accuracy, and offers experimental results of hardware implementation on an FPGA.

4.1 Accuracy

Accuracy of the previous and proposed SPFP approximate multipliers is compared in Table 3. For evaluation, 200 million multiplications on random SPFP numbers between 0 and 1 are executed as presented in [5]. Rows represent different multiplication algorithms; columns show the percentage of these 200 million multiplications accurate within the specified tolerance (denoted as Correct (%)) as presented in [6]. We compare against the conventional Mitchell [8] and the iterative Mitchell [6] in this experiment. Originally, the iterative Mitchell is designed for integers, but extended to an SPFP multiplier in this section. There are three proposed designs in Table 3: baseline (denoted as BASE), baseline with the rounding scheme (denoted as RND), and baseline with the rounding scheme and the operand selection scheme (denoted as SEL) where input mantissas are truncated by 11-bits (denoted as TRNC). As shown in the second row, the conventional Mitchell shows low accuracy. On the other hand, the iterative Mitchell algorithm significantly improves the accuracy as the number of iterations increases. As shown in the fifth, ninth, and thirteenth rows, our rounding scheme significantly improves the accuracy compared with when it is not applied. Our operand selection scheme further enhances the accuracy. Specifically, the rates satisfying Error < 0.1% of which iteration numbers are 2, 3, and 4 are increased by 87%, 52%, and 17%, respectively.

Table 4 shows experimental results to select the type and input bit-width of the prediction table. The third, fourth, and fifth columns

Table 4: Prediction accuracy and correct percentage (Error < 0.5%) depending on the type and size of the prediction table

	Input size	Pred. accuracy (%)			Correct (%)		
		n = 2	n = 3	n = 4	n = 2	n = 3	n = 4
Pred. Table (n=2)	12	<u>95.28</u>	79.63	77.09	32.16	83.01	99.81
	8	95.14	79.57	77.05	32.16	83.01	99.81
	7	94.77	79.38	76.96	32.12	83.01	99.81
	6	93.64	78.70	76.65	31.36	83.01	99.81
Pred. Table (n=3)	12	79.70	<u>95.77</u>	82.68	30.68	88.13	99.90
	8	75.20	91.99	81.74	27.22	88.13	99.90
	7	69.45	85.69	78.63	23.55	87.91	99.90
	6	60.11	74.15	72.63	20.00	82.31	99.90
Pred. Table (n=4)	12	73.59	82.63	<u>90.81</u>	28.98	83.90	99.99
	8	55.75	58.36	72.12	19.18	71.66	99.99
	7	50.49	52.24	62.57	17.95	66.55	99.92
	6	50.16	51.87	59.15	17.68	66.04	99.24

of this table show prediction accuracy compared to the optimal selection. The last three columns present the correct percentage satisfying Error < 0.5%. As represented by underlines, using their own prediction table and 12-bit input shows the best prediction accuracy. When a single prediction table is used and an additional truncation to this table is applied, using the prediction table with n of 2 and 5-bit truncation shows reasonable results in general. Therefore, we choose them in the proposed design.

4.2 Hardware Implementation Results

Table 5 compares hardware implementation results of the iterative Mitchell [6] and the proposed multipliers (BASE, BASE+RND, BASE+RND+TRNC, and BASE+RND+TRNC+SEL). As described in Section 4.1, the iterative Mitchell is extended to an SPFP multiplier. Its architecture is similar to ours. However, the number of shift amount calculators is two (2 LODs and 2 shifters for the mantissa). In addition, the shift-based multiplier has 5-stages: Stage1 to wait

Table 5: FPGA (Virtex UltraScale) implementation results

	Iterative Mitchell [6]	Proposed			
		BASE	+RND	+TRNC	+SEL
No. LOD(s)	2	1	1	1	1
No. Adder(s)	2	1	1	1	1
No. Shifters	5	2	2	2	2
$f_{\max}$ (MHz)	450	510	440	560	500
LUT	685	317	397	237	308
FF	241	142	144	100	100
Latency	$n+4$	$n+2$	$n+2$	$n+2$	$n+2$

for shift amounts; Stage2 to calculate the second and third terms of (5), which requires 2 shifters for the mantissa; Stage3 to sum up the two terms (1 adder for the mantissa); Stage4 to normalize the sum to the first term (1 shifter for the mantissa); and Stage5 to accumulate the final output (1 adder for the mantissa).

All previous and proposed designs are implemented in Verilog HDL, and synthesized and placed-and-routed using Xilinx Vivado Design Suite (2016.4). The target device is Xilinx Virtex UltraScale FPGA (xcvu190-flgc2104-2-e) that has 1,074,240 look up tables (LUTs) and 2,148,480 flip flops (FFs). As shown in the second, third, and fourth rows, our proposed designs use one fewer LOD and adder for the mantissa, and three fewer shifters for the mantissa than the iterative Mitchell. The fifth, sixth, and seventh rows present the maximum frequency, the numbers of LUTs and FFs, respectively. Compared with the iterative Mitchell, our proposed design with the rounding scheme decreases the numbers of LUTs and FFs by about 42% and 40% at the similar frequency, respectively. As shown in the fourth and fifth columns, the 11-bit truncation increases the maximum frequency from 440MHz to 560MHz, and reduces the numbers of LUTs and FFs by 40% and 31%, respectively. The operand selector for higher accuracy re-increases the critical path delay and the number of LUTs and FFs. However, compared to the iterative Mitchell, the maximum frequency is 50MHz higher, which assures a higher throughput, and our design uses only 45% of LUTs and 41% of FFs. The eighth row of Table 5 shows the latency. Since all designs in Table 5 are based on iteration, they can trade off latency against accuracy without changing logic blocks. The latency of our proposed design is two cycles shorter than that of the iterative Mitchell. Note that the truncation scheme is not used in the original iterative Mitchell. However, applying the 11-bit truncation reduces the number of LUTs and FFs. For example, assuming that its hardware resources are reduced at the same rate as our design, it uses 409 LUTs and 167 FFs, which means that it needs 33% more LUTs, 67% more FFs, and two cycles longer latency than our work (BASE+RND+TRNC+SEL).

Our design shows lower accuracy than the iterative Mitchell. However, in applications which have a long interval of input data (e.g., 4 cycles), we can reduce the difference in accuracy by increasing the number of iterations while having the better hardware properties. Fig. 5 shows normalized accuracy, latency, and hardware resource comparison when n values are 2, 3, and 4. Since the normalized rates of LUT and FF are not changed, they are only presented in Fig. 5(c). The normalized correct percentages satisfying Errors < 0.5% of our design becomes close to that of the iterative

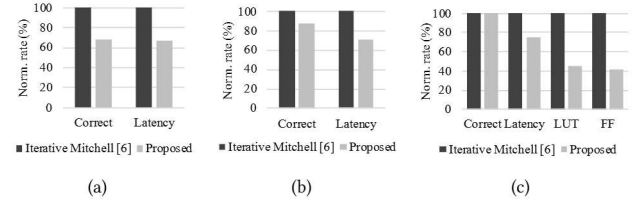


Figure 5: Normalized accuracy (Error < 0.5%), latency, and hardware resource comparison. (a) n is 2, (b) n is 3, (c) n is 4.

Mitchell as the n value increases while our design only uses about 43% of the hardware resources and shows about 71% of the latency.

5 CONCLUSION

This paper proposes an area-efficient SPFP multiplier architecture that can trade off accuracy against latency. We did not consider using digital signal processing (DSP) slices in this paper. Typically, the number of DSP slices is much more limited than those of LUTs and FFs on FPGAs. In systems where an extremely large number of multipliers are needed or other operators exploit DSP slices, a design only using LUTs and FFs is useful. Compared with the Xilinx LogiCore IP block for an SPFP multiplier [12], our design reduces the numbers of LUTs and FFs by 46% and 81%, respectively, and improves the maximum frequency by 19% when the pipeline depth of the LogiCore block is five and DSP slices are not utilized. Of course, this comparison is not completely fair because the LogiCore block is not an approximate and iterative design. Nevertheless, it implies that our design can be efficiently utilized in some applications where hardware resources are limited and approximation errors are tolerated. In future work, we hope to apply our iterative multiplier to applications that can be improved when the iteration number is allowed to be adaptively adjusted.

REFERENCES

- [1] M. S. Kim, A. A. Del Barrio, R. Hermida, and N. Bagherzadeh, "Low-Power Implementation of Mitchell's Approximate Logarithmic Multiplication for Convolutional Neural Networks," in *Proc. ASP-DAC*, Jan. 2018.
- [2] M. Imani, D. Peroni, and T. Rosing, "CFPU: Configurable Floating Point Multiplier for Energy-Efficient Computing," in *Proc. DAC*, Jun. 2017.
- [3] P. Yin, C. Wang, W. Liu, and F. Lombardi, "Design and Performance Evaluation of Approximate Floating-Point Multipliers," in *Proc. ISVLSI*, Jul. 2016.
- [4] S. Hashemi, R. Iris Bahar, and S. Reda, "DRUM: A Dynamic Range Unbiased Multiplier for Approximate Applications," in *Proc. ICCAD*, Nov. 2015.
- [5] H. Zhang, W. Zhang, and J. Lach, "A Low-Power Accuracy-Configurable Floating Point Multiplier," in *Proc. ICCD*, Dec. 2014.
- [6] Z. Babic, A. Avramovic, and P. Bulic, "An Iterative Logarithmic Multiplier," *Elsevier Microprocessors and Microsystems*, vol. 35, no. 1, pp. 23-33, Feb. 2011.
- [7] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, Dec. 2017.
- [8] J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms," *IRE Trans. Electronic Computers*, vol. EC-11, no. 4, pp. 512-517, Aug. 1962.
- [9] IEEE, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2008*, pp. 1-58, Aug. 2008.
- [10] M. Al-Ashrafy, A. Salem, and W. Anis, "An Efficient Implementation of Floating Point Multiplier," in *Proc. SIECP*, Apr. 2011.
- [11] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing Power by Optimizing the Necessary Precision/Range of Floating-point Arithmetic," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 8, no. 3, pp. 273-286, Jun. 2000.
- [12] Xilinx, "LogiCORE IP Floating-Point Operator v7.1 Product Guide (PG060)," [Online] https://www.xilinx.com/support/documentation/ip_documentation/floating_point/v7_1/pg060-floating-point.pdf.